

Practical Assignment 1 APA 2025

1st Francesco De Nicola

*Department Of Electrical and Computer Engineering
University of Coimbra (Erasmus Program)*

Coimbra, Portugal

uc2025203419@student.uc.pt

2nd Samuel Bayowa Edema

*Department Of Electrical and Computer Engineering
University Of Coimbra*

Coimbra, Portugal

uc2024133618@student.uc.pt

Abstract—This report presents our implementation and evaluation of convolutional neural networks for image classification tasks. We implemented a custom CNN architecture for CIFAR-10 classification, achieving 85.47% test accuracy and mean-class accuracy. Additionally, we evaluated ResNet18 and MobileNetV3-Small architectures on STL-10, comparing training from scratch versus using pretrained ImageNet weights. We also implemented a middle-fusion strategy combining both architectures, achieving 93.33% mean-class accuracy on STL-10. All implementations were developed in PyTorch with comprehensive data augmentation, regularization techniques, and systematic hyperparameter tuning.

Index Terms—Convolutional Neural Network, Image Classification, CIFAR-10, STL-10, Transfer Learning, Model Fusion, ResNet, MobileNet.

I. INTRODUCTION

Image classification remains a fundamental task in computer vision with applications ranging from autonomous systems to medical diagnosis. This assignment explores two complementary approaches: designing a custom CNN architecture for a specific dataset and leveraging pretrained models through transfer learning. We implemented these approaches on two benchmark datasets: CIFAR-10 and STL-10.

The assignment is structured in two parts: Part I focuses on building and optimizing a custom CNN for CIFAR-10, while Part II evaluates standard architectures (ResNet18 and MobileNetV3-Small) on STL-10 using both scratch and pretrained training paradigms, culminating in a middle-fusion strategy that combines features from multiple models.

II. PART I: CUSTOM CNN ON CIFAR-10

A. Dataset and Preprocessing

The CIFAR-10 dataset contains 60,000 color images of size 32×32 pixels distributed across 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). We split the original 50,000 training images into 40,000 for training and 10,000 for validation using an 80/20 split with random shuffling. The test set of 10,000 images remained unchanged for final evaluation.

We computed the normalization statistics directly from the CIFAR-10 training dataset rather than using hardcoded values. We iterated through the entire training set to calculate the per-channel mean and standard deviation, obtaining mean = (0.4914, 0.4822, 0.4465) and std = (0.2470, 0.2435, 0.2616).

This ensures that our normalization accurately reflects the dataset distribution.

We implemented comprehensive data augmentation for the training set to improve generalization and reduce overfitting. The augmentation pipeline included random horizontal flips with probability 0.5, random crops with padding of 4 pixels, and random erasing with probability 0.25. For validation and testing, we applied only normalization without augmentation to ensure consistent evaluation.

We set a random seed of 48 across NumPy and PyTorch to ensure reproducibility of results across different runs.

B. Implementation Pipeline

Figure 1 illustrates the complete implementation pipeline for Part I, showing the sequential steps from data loading through evaluation.

C. Model Architecture

We designed a custom CNN architecture consisting of three convolutional blocks followed by global average pooling and a three-layer classifier. Figure 2 illustrates the complete architecture with explicit representation of the layer groupings.

The first block (conv1+2) processes the 3-channel RGB input through two 3×3 convolutional layers, each outputting 64 feature maps with batch normalization and ReLU activation. After the second convolution, we apply 2×2 max pooling with stride 2, reducing spatial resolution from 32×32 to 16×16, followed by spatial dropout with probability 0.25 to prevent overfitting.

The second block (conv3+4) contains two convolutional layers expanding from 64 to 128 channels. The first layer (conv3) increases the channel depth from 64 to 128, while the second layer (conv4) maintains 128 channels. After batch normalization and ReLU activation, max pooling reduces resolution from 16×16 to 8×8, again followed by dropout.

The third block (conv5+6) further increases representational capacity with two layers operating at 256 channels. Conv5 expands from 128 to 256 channels, and conv6 maintains this depth. The final max pooling reduces spatial dimensions to 4×4, followed by dropout before the classification head.

Instead of flattening the 256×4×4 feature maps directly, we applied adaptive average pooling (AdaptiveAvgPool2d) to obtain a compact 256-dimensional feature vector (1×1×256). This design choice reduces the number of parameters in the

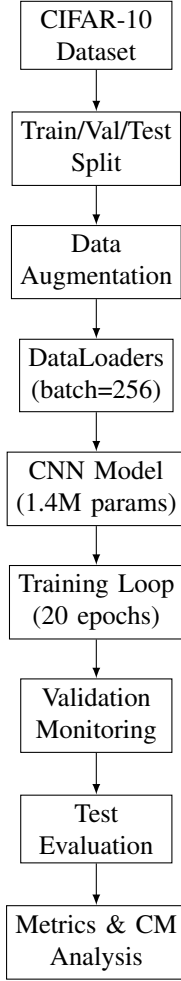


Fig. 1. Implementation pipeline for Part I showing the sequential stages from data loading through evaluation on CIFAR-10

fully connected layers and provides translation invariance, making the model more robust to spatial variations in the input.

The classifier consists of three fully connected layers with the following structure: fc1 projects from 256 to 512 dimensions, fc2 reduces from 512 to 256 dimensions, and fc3 maps to the final 10 output classes. We applied batch normalization and ReLU activation after fc1 and fc2, with dropout ($p=0.5$) between layers to prevent overfitting in the classification head.

We initialized all convolutional layer weights using Kaiming normal initialization, which is particularly effective for ReLU activations by accounting for the variance of the activation function. Batch normalization layers were initialized with weights of 1 and biases of 0, while linear layer weights were initialized from a normal distribution with standard deviation 0.01. The complete architecture contains 1,414,218 trainable parameters.

D. Training Configuration

We trained the model for 20 epochs using a batch size of 256. We used the Adam optimizer with initial learning rate of 1×10^{-3} , weight decay of 1×10^{-3} , and default momentum

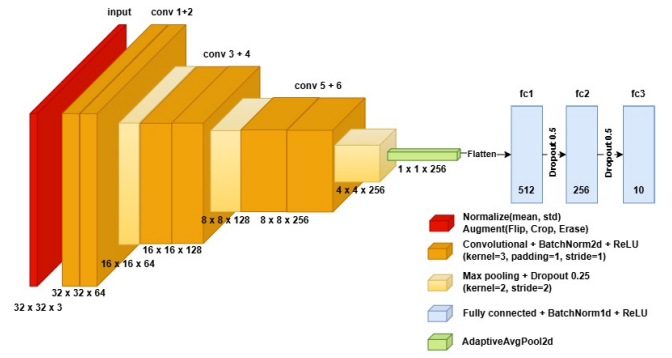


Fig. 2. Custom CNN architecture for CIFAR-10 classification. The network consists of three convolutional blocks labeled as conv1+2 (64 channels), conv3+4 (128 channels), and conv5+6 (256 channels), where each label represents two sequential 3×3 convolutional layers with batch normalization and ReLU activation. Spatial dimensions are progressively reduced through 2×2 max pooling with stride 2 ($32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8 \rightarrow 4 \times 4$), followed by adaptive global average pooling that reduces to $1 \times 1 \times 256$ features, and finally a three-layer fully connected classifier ($512 \rightarrow 256 \rightarrow 10$) with dropout regularization.

parameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$). The learning rate was scheduled using cosine annealing, decreasing from the initial value to a minimum of 1×10^{-6} over the 20 epochs, which helps the model converge to better local minima.

We implemented label smoothing with a smoothing factor of 0.1 to regularize the model and prevent overconfident predictions. Instead of using hard targets (one-hot encoded labels), label smoothing distributes a small amount of probability mass to incorrect classes, which improves calibration and generalization.

We incorporated early stopping with a patience of 12 epochs, monitoring validation accuracy to save the best model checkpoint. This prevents overfitting by halting training when validation performance stops improving.

E. Hyperparameter Tuning

To understand the impact of optimization hyperparameters on model performance, we systematically compared four different configurations of learning rate and weight decay. We trained separate models with the following combinations:

TABLE I
HYPERPARAMETER COMPARISON ON CIFAR-10 VALIDATION SET
SHOWING IMPACT OF LEARNING RATE AND WEIGHT DECAY ON MODEL PERFORMANCE

Configuration	Val Acc (%)	mCA	Precision	Recall	F1
lr= $1e-3$, wd= $5e-4$	84.52	0.8456	0.8451	0.8456	0.8440
lr= $5e-4$, wd= $5e-4$	79.71	0.7976	0.7961	0.7976	0.7949
lr= $1e-3$, wd= $1e-3$	85.33	0.8538	0.8527	0.8538	0.8526
lr= $1e-4$, wd= $1e-6$	58.56	0.5866	0.5940	0.5866	0.5811

The configuration with learning rate 1×10^{-3} and weight decay 1×10^{-3} achieved the best validation accuracy of 85.33%, demonstrating that higher weight decay provides better regularization for this architecture. This result indicates that

L2 regularization is crucial for preventing overfitting on the custom CNN.

The low learning rate configuration (1×10^{-4}) significantly underperformed with only 58.56% validation accuracy, clearly showing severe underfitting. The learning rate was too small to effectively train the network within 20 epochs, resulting in the model failing to converge from random initialization.

The moderate learning rate with lower weight decay ($lr=1e-3$, $wd=5e-4$) achieved 84.52%, showing competitive performance but slightly worse than the higher weight decay variant (0.81 percentage point difference). This 0.81 point improvement with higher weight decay suggests that the model with $wd=1e-3$ achieves better generalization through stronger regularization.

Comparing configurations with $lr=1e-3$, we observe that reducing weight decay from $1e-3$ to $5e-4$ results in overfitting risk, as the model becomes less regularized. The configuration with $wd=1e-3$ provides optimal regularization strength without over-constraining the model parameters.

We selected the best hyperparameter configuration ($lr=1e-3$, $wd=1e-3$) for our final model reported in subsequent sections.

F. Training Dynamics

Figure 3 shows the training dynamics over 20 epochs. The training loss per batch exhibits a steady decrease with minor fluctuations, indicating stable gradient descent. The average training loss per epoch decreases from approximately 1.95 to 0.97, while validation loss decreases from 1.70 to 0.89, showing good convergence without significant overfitting. Validation accuracy increases steadily from approximately 43% in epoch 2 to over 85% by epoch 20, demonstrating effective learning. The slight gap between training and validation loss indicates that our regularization strategies (dropout, weight decay, label smoothing) successfully prevented overfitting.

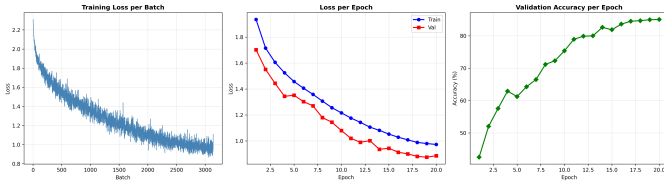


Fig. 3. Training dynamics for CIFAR-10: loss per batch (left), average loss per epoch (center), and validation accuracy per epoch (right). The model converges smoothly with validation accuracy steadily increasing while maintaining moderate validation loss.

Figure 4 compares the training and validation loss curves across all four hyperparameter configurations. The configuration with $lr=1e-4$, $wd=1e-6$ (red curves) shows significantly higher loss values and slower convergence, confirming that the learning rate was too conservative for effective training. The three configurations with $lr=1e-3$ show similar convergence patterns, with the variant using $wd=1e-3$ (green) achieving slightly lower validation loss by the final epochs, validating our hyperparameter selection.

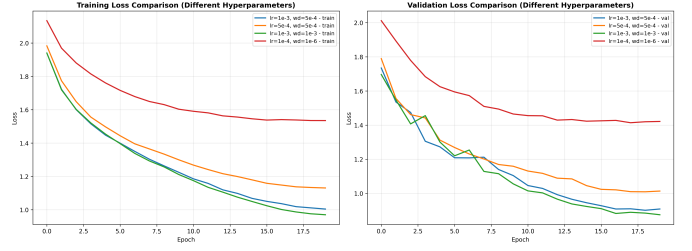


Fig. 4. Hyperparameter comparison showing training loss (left) and validation loss (right) for four different configurations. The red curves ($lr=1e-4$, $wd=1e-6$) demonstrate underfitting with high loss throughout training, while green curves ($lr=1e-3$, $wd=1e-3$) show optimal convergence.

G. Test Set Evaluation

We evaluated the best model (trained with $lr=1e-3$, $wd=1e-3$) on the held-out test set of 10,000 images. The model achieved the following metrics:

TABLE II
FINAL TEST SET PERFORMANCE ON CIFAR-10

Metric	Value
Accuracy	0.8547
Mean-Class Accuracy (mCA)	0.8547
Precision (macro)	0.8544
Recall (macro)	0.8547
F1-score (macro)	0.8526

The model achieved 85.47% test accuracy, exceeding the assignment requirement of 82%. The mean-class accuracy of 85.47% indicates balanced performance across all 10 classes. The macro-averaged precision, recall, and F1-score are all approximately 85%, demonstrating that the model performs consistently across classes without significant bias toward any particular category.

H. Confusion Matrix Analysis

Figure 5 presents the confusion matrix on the CIFAR-10 test set. The diagonal elements show the correctly classified instances, with most classes achieving 757-947 correct predictions out of 1,000 test samples per class. Automobiles and trucks are classified with the highest accuracy (947 and 934 correct predictions respectively), while cats and birds show relatively more confusion (619 and 757 correct predictions respectively).

Common misclassifications include cats confused with dogs (165 instances), which are semantically related categories. Birds are also frequently confused with airplanes (44 instances), likely due to their presence in sky backgrounds. Cats are confused with other animal classes, reflecting similar textures and poses. Overall, the confusion matrix reveals that most errors occur between semantically or visually similar classes, which is expected behavior.

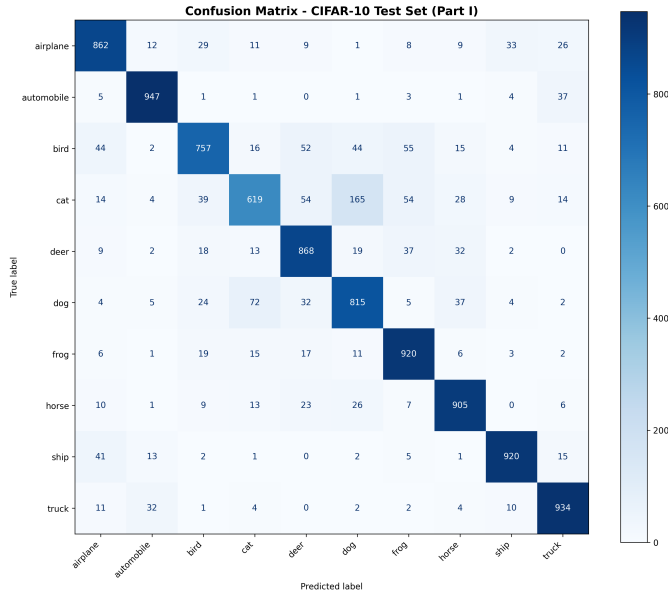


Fig. 5. Confusion matrix for CIFAR-10 test set showing predictions across 10 classes

III. PART II: TRANSFER LEARNING AND FUSION ON STL-10

A. Dataset and Preprocessing

We used the STL-10 dataset, which contains higher-resolution images (96×96 pixels) across the same 10 classes as CIFAR-10. The training set contains 5,000 labeled images, which we split 80/20 into 4,000 training and 1,000 validation samples. The test set contains 8,000 images for final evaluation.

We implemented two different preprocessing pipelines to accommodate both scratch and pretrained training paradigms. For training from scratch, we used the native 96×96 resolution with only ToTensor conversion, allowing models to learn directly from the STL-10 image characteristics. For pretrained models, we resized images to 224×224 pixels to match ImageNet training dimensions, applying random resized crops and horizontal flips during training, and center crops during evaluation. We normalized pretrained images using ImageNet statistics (mean = (0.485, 0.456, 0.406), std = (0.229, 0.224, 0.225)).

B. Model Architectures and Training Protocol

We evaluated two standard architectures: ResNet18 and MobileNetV3-Small. ResNet18 is a residual network with 18 layers featuring skip connections that enable training of deeper networks. MobileNetV3-Small is an efficient architecture designed for mobile and embedded devices, using depthwise separable convolutions to reduce computational cost while maintaining reasonable accuracy.

We modified both architectures for STL-10 by replacing the final classification layer with a new linear layer outputting 10 classes. For pretrained models, we initialized all layers except the final classifier with ImageNet weights (ImageNet

pretraining was performed on 1,000 classes; we fine-tuned only the last layer while keeping backbone weights fixed). For scratch training, we used random initialization for all weights.

We used consistent hyperparameters across all Part II experiments to enable fair comparison. We trained all models for 30 epochs using batch size 64, learning rate 1×10^{-3} , weight decay 5×10^{-4} , and early stopping patience of 10 epochs. We applied the same label smoothing (0.1) and cosine annealing scheduler (minimum LR 1×10^{-6}) as in Part I.

The smaller batch size (64 vs 256 in Part I) was chosen because STL-10 has higher resolution images and fewer training samples, making smaller batches more appropriate for generalization. We used GPU acceleration for all training runs. We set the same random seed (48) across all experiments to ensure reproducibility.

C. Results: Training from Scratch

Training both architectures from scratch on STL-10 proved challenging due to the limited training data (only 4,000 samples). ResNet18 achieved 62.6% test accuracy and mean-class accuracy, significantly outperforming MobileNetV3-Small which achieved only 42.55%. This substantial performance gap suggests that MobileNetV3’s aggressive parameter reduction makes it more difficult to train from scratch with limited data.

TABLE III
TEST SET PERFORMANCE FOR SCRATCH TRAINING ON STL-10

Model	Accuracy	mCA	Precision	Recall	F1
ResNet18	0.6260	0.6260	0.6269	0.6260	0.6255
MobileNetV3-S	0.4255	0.4255	0.4745	0.4255	0.4370

Figures 6 show the confusion matrices for both models trained from scratch. ResNet18 demonstrates more balanced performance across classes, while MobileNetV3-Small shows severe confusion, particularly struggling with several classes and often making incorrect predictions. This reflects insufficient model capacity or inadequate training with the limited dataset.

Figure 7 shows the training curves for both models. ResNet18 achieves smooth convergence with validation accuracy steadily increasing to approximately 60%. MobileNetV3-Small shows more erratic validation behavior and plateaus at much lower accuracy, with significant oscillation suggesting difficulty in optimization.

D. Results: Transfer Learning with Pretrained Models

Using ImageNet pretrained weights dramatically improved performance for both architectures. ResNet18 achieved 93.15% test accuracy, a massive 30.55 percentage point improvement over scratch training. MobileNetV3-Small reached 72.6%, a 30.05 percentage point gain compared to training from scratch. These results demonstrate the power of transfer learning, especially when training data is limited.

ResNet18 pretrained significantly outperformed MobileNetV3-Small pretrained, likely because ResNet18’s

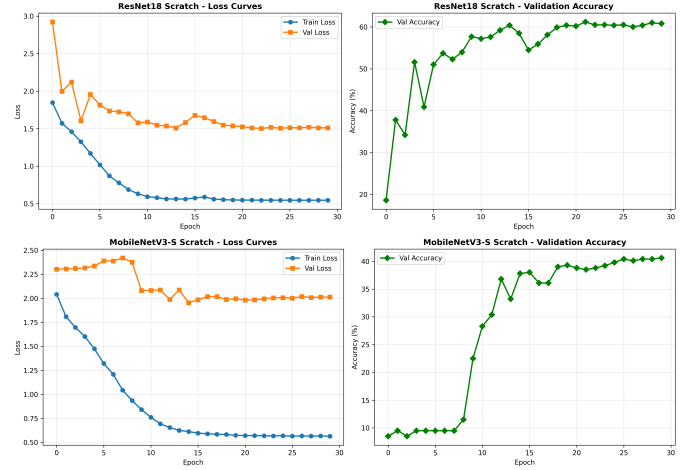
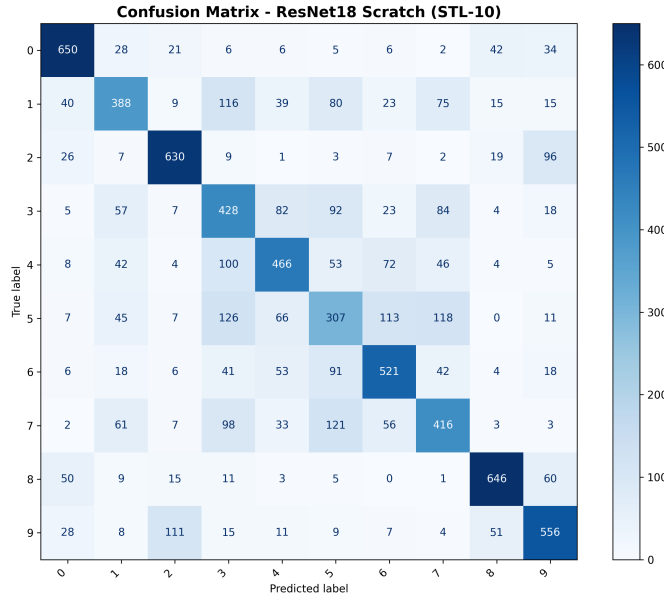


Fig. 7. Training and validation curves for scratch models on STL-10: ResNet18 (top), MobileNetV3-Small (bottom)

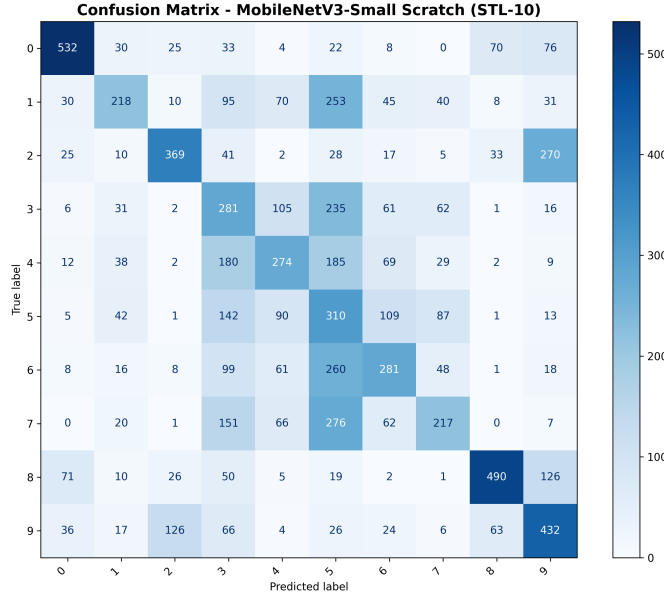


Fig. 6. Confusion matrices for models trained from scratch on STL-10: ResNet18 (top), MobileNetV3-Small (bottom)

TABLE IV
TEST SET PERFORMANCE FOR PRETRAINED MODELS ON STL-10

Model	Accuracy	mCA	Precision	Recall	F1
ResNet18	0.9315	0.9315	0.9320	0.9315	0.9315
MobileNetV3-S	0.7260	0.7260	0.8099	0.7260	0.7287

larger capacity (approximately 11M parameters) better preserves discriminative ImageNet features compared to MobileNetV3-Small's compressed architecture (approximately 2.5M parameters). The macro precision for MobileNetV3-Small (0.810) is notably higher than recall (0.726), suggesting the model is more conservative in its predictions.

Figure 8 and Figure 9 show the confusion matrices for

pretrained models. ResNet18 achieves excellent classification with most diagonal elements above 740 out of 800 per class, demonstrating balanced performance. MobileNetV3-Small shows more confusion across several classes, though still substantially better than its scratch-trained counterpart.

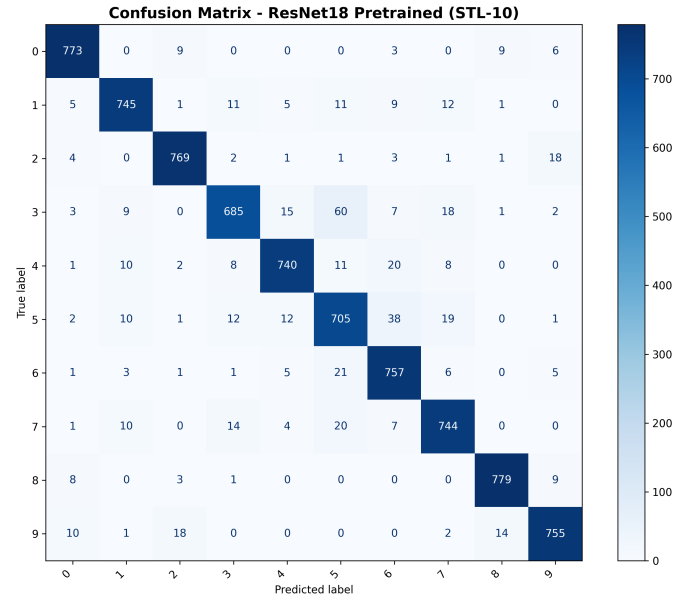


Fig. 8. Confusion matrix for pretrained models on STL-10: ResNet18

Figure 10 shows training curves for pretrained models. ResNet18 demonstrates very stable training with validation accuracy quickly reaching and maintaining high levels (around 90-93%). MobileNetV3-Small shows more variability but still achieves reasonable convergence. The validation loss for ResNet18 exhibits minor fluctuations but remains low, while MobileNetV3-Small shows higher and more variable validation loss.

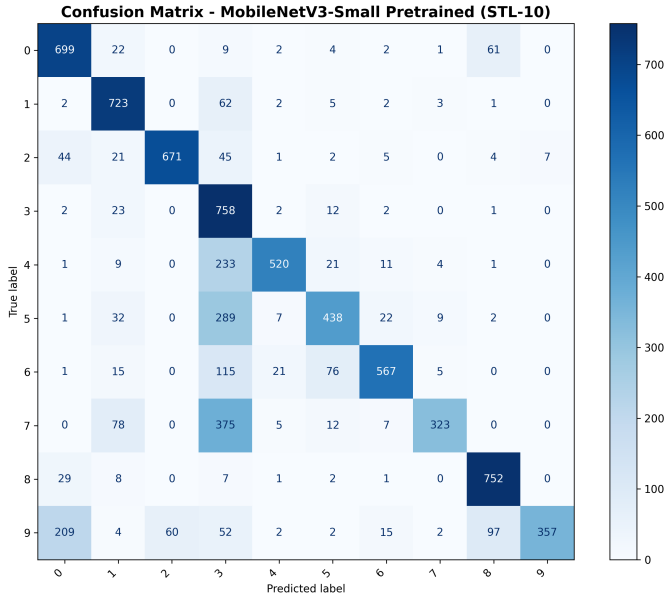


Fig. 9. Confusion matrices for pretrained models on STL-10: MobileNetV3-Small

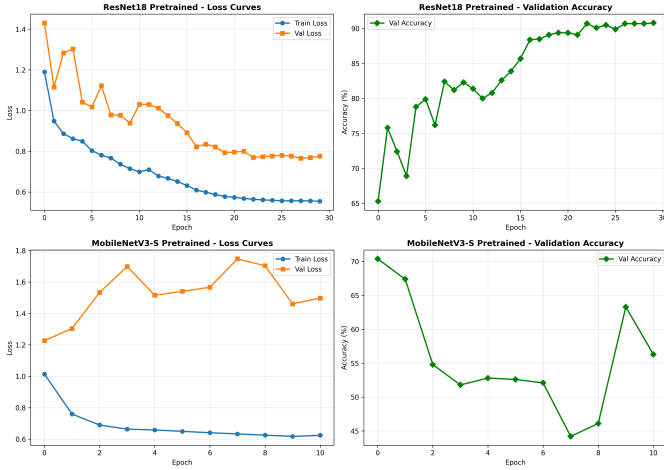


Fig. 10. Training and validation curves for pretrained models on STL-10: ResNet18 (top), MobileNetV3-Small (bottom)

E. Middle Fusion Strategy and Architecture

We implemented a middle-fusion approach that combines feature representations from both pretrained models before the final classification decision. Figure 11 illustrates the middle fusion architecture.

We selected the best-performing variant of each architecture (both pretrained) as feature extractors. We froze all parameters in both backbones to preserve their learned ImageNet features, only fine-tuning the fusion classifier.

ResNet18 outputs 512-dimensional features after global average pooling, while MobileNetV3-Small produces 576-dimensional features. We concatenated these representations to obtain a 1088-dimensional joint feature vector. We then trained a new classification head consisting of a linear layer projecting

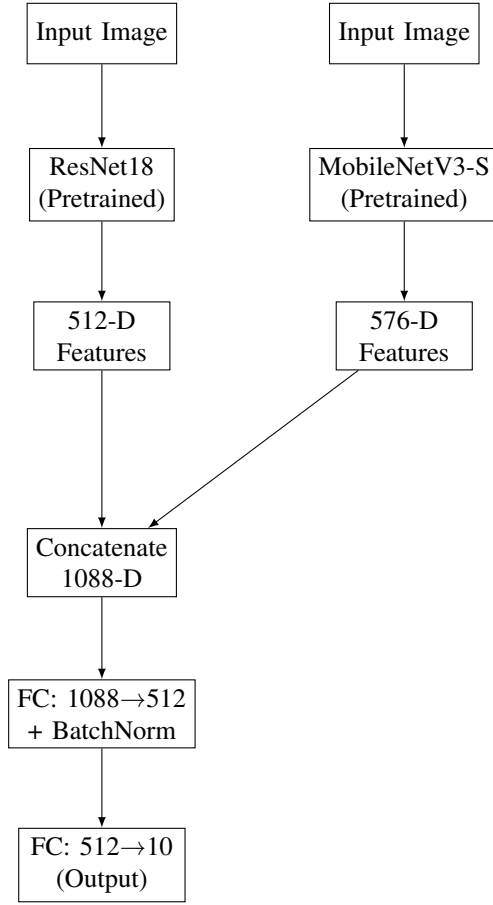


Fig. 11. Middle fusion architecture combining ResNet18 and MobileNetV3-Small feature representations. Both frozen backbones extract 512-D and 576-D features respectively, which are concatenated to 1088-D and processed by a trainable classifier.

to 512 dimensions, batch normalization, ReLU activation, dropout (0.5), and a final linear layer mapping to 10 classes.

We trained only the classification head parameters while keeping both feature extractors frozen, using the same hyper-parameters as individual model training (batch size 64, learning rate 1×10^{-3} , weight decay 5×10^{-4} , 30 epochs maximum with early stopping patience 10). This approach ensures that we only learn how to effectively combine the complementary features from both models without modifying their pre-learned representations.

F. Fusion Results and Analysis

The fusion model achieved 93.33% test accuracy and mean-class accuracy, slightly exceeding ResNet18's individual performance and substantially outperforming MobileNetV3-Small alone. This result is particularly noteworthy because the fusion model leverages complementary features from both architectures.

While the fusion model's accuracy marginally exceeds ResNet18 alone (93.33% vs 93.15%), combining both feature extractors provides a slight advantage. The fusion model achieves the best overall precision (0.9334) and maintains

TABLE V
TEST SET PERFORMANCE FOR MIDDLE FUSION ON STL-10

Model	Accuracy	mCA	Precision	Recall	F1
Middle Fusion	0.9333	0.9333	0.9334	0.9333	0.9332

excellent F1 score (0.9332), indicating slightly more balanced predictions. The MobileNetV3-Small features contribute complementary information that provides a small but measurable boost beyond ResNet18’s strong baseline.

Figure 12 shows the confusion matrix for the fusion model. The classification pattern demonstrates excellent performance across all classes (most diagonal elements above 740 out of 800). This confirms that the fusion successfully leverages the strong feature representations from both models.

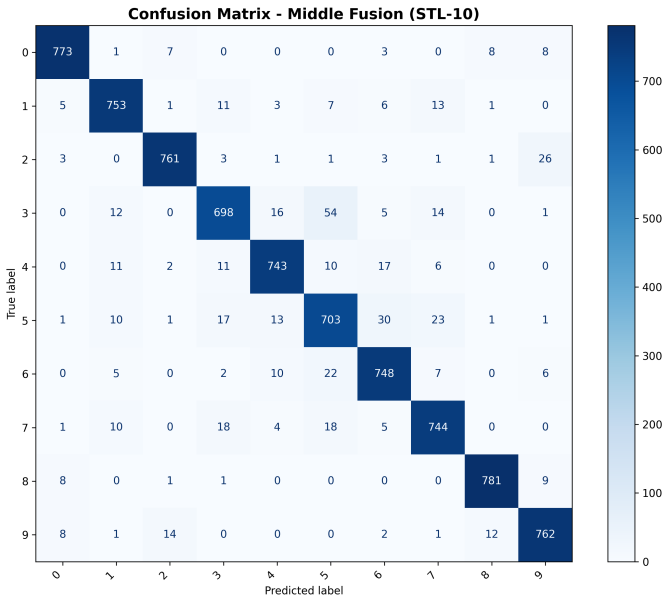


Fig. 12. Confusion matrix for middle fusion model on STL-10

Figure 13 shows the training dynamics for the fusion model. Both training and validation losses decrease smoothly and converge to low values, indicating stable optimization. Validation accuracy rises quickly and stabilizes around 91% with minor fluctuations. The training curves demonstrate that the fusion classification head successfully learned to combine features from both frozen backbones.

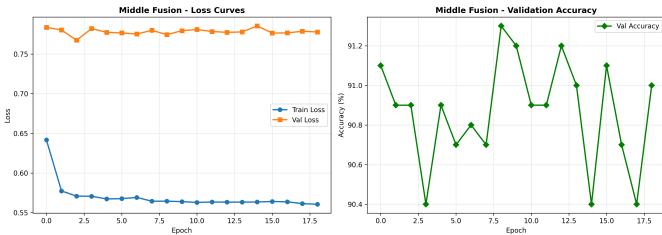


Fig. 13. Training and validation curves for middle fusion model on STL-10

G. Comparative Analysis

TABLE VI
TEST SET PERFORMANCE FOR MIDDLE FUSION ON STL-10

Model Configuration	Accuracy	mCA	Precision	Recall	F1
ResNet18 scratch	0.6260	0.6260	0.6269	0.6260	0.6255
ResNet18 pretrained	0.9315	0.9315	0.9320	0.9315	0.9315
MobileNetV3-S scratch	0.4255	0.4255	0.4745	0.4255	0.4370
MobileNetV3-S pretrained	0.7260	0.7260	0.8099	0.7260	0.7287
Middle Fusion	0.9333	0.9333	0.9334	0.9333	0.9332

Table VI summarizes all Part II results. The comparison reveals several key insights:

Transfer learning provides dramatic performance gains for both architectures. ResNet18 improved by 30.55 percentage points, while MobileNetV3-Small improved by 30.05 percentage points when using pretrained weights. This demonstrates that ImageNet pretraining effectively captures generalizable visual features applicable to STL-10.

ResNet18 consistently outperformed MobileNetV3-Small in both scratch and pretrained settings, suggesting that ResNet’s larger capacity and residual connections provide better feature learning and transfer. The middle fusion strategy successfully combined both models and achieved the best overall performance, demonstrating that combining complementary features can provide measurable improvements.

Key Comparative Observations:

1. **Scratch Training Limitations:** Both models struggled when trained from scratch with only 4,000 samples, achieving 62.6% and 42.55% respectively. This demonstrates the severe data limitations of training state-of-the-art architectures without substantial datasets.

2. **Transfer Learning Effectiveness:** The 30-percentage-point improvements achieved by both models using ImageNet pre-training highlight the extraordinary power of transfer learning. ImageNet features generalize remarkably well to STL-10 despite the domain differences.

3. **Architecture Efficiency Trade-offs:** MobileNetV3-Small’s aggressive parameter reduction (2.5M vs 11M) comes at the cost of representational capacity. While pre-trained MobileNetV3-Small (72.6%) underperforms ResNet18 (93.15%), it remains a viable option for resource-constrained scenarios.

4. **Fusion Benefits:** The fusion model’s 0.18-point improvement over ResNet18 alone suggests that MobileNetV3-Small captures complementary information despite its lower individual performance. The fusion approach successfully exploits this complementarity.

5. **Regularization and Stability:** ResNet18 demonstrated more stable training curves with ResNet18 pretrained showing minimal validation loss fluctuation, while MobileNetV3-Small showed more erratic behavior, particularly in scratch training.

Figure 14 illustrates the mean-class accuracy comparison across all models, clearly showing the dramatic impact of transfer learning and the slight advantage provided by the fusion approach.

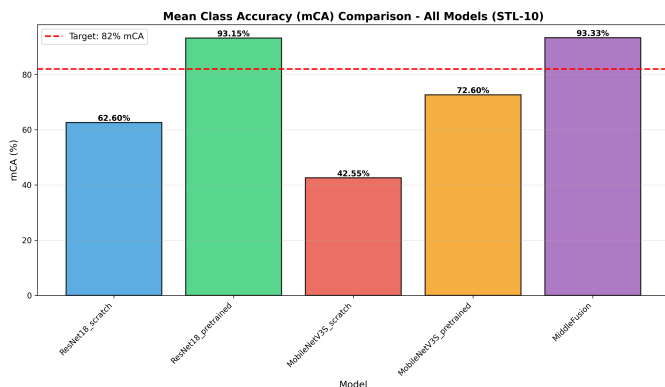


Fig. 14. Mean Class Accuracy (mCA) comparison across all STL-10 models showing dramatic improvements from transfer learning and marginal gains from fusion strategy. Target of 82% mCA is achieved by pretrained models and fusion.

IV. DISCUSSION

Our experiments demonstrate the effectiveness of convolutional neural networks for image classification across different scales and training paradigms. The custom CNN achieved strong performance on CIFAR-10 through careful architecture design, extensive data augmentation, and appropriate regularization. Computing normalization statistics directly from the data rather than using hardcoded values ensured accurate preprocessing. Hyperparameter tuning revealed that higher weight decay (1×10^{-3}) improved generalization, while insufficient learning rates (1×10^{-4}) prevented effective training.

The STL-10 experiments highlighted the critical importance of transfer learning when training data is limited. Pretrained ImageNet weights provided dramatic improvements (30+ percentage points) for both ResNet18 and MobileNetV3-Small. This validates the common practice of using pretrained models as feature extractors in low-data regimes.

The middle-fusion strategy demonstrated that combining multiple architectures can be effective, with the fusion model achieving 93.33% mCA compared to 93.15% for ResNet18 alone. While the improvement is modest, it demonstrates that leveraging complementary features from different architectures can provide measurable benefits. Future fusion strategies could explore weighted combinations, attention mechanisms, or training with different objectives to better leverage complementary features.

Limitations of our work include the relatively small training set for STL-10 (4,000 samples), which may not fully demonstrate the potential of training from scratch. Additionally, we used consistent hyperparameters across all Part II experiments; architecture-specific tuning might improve individual model performance. The fusion model could potentially benefit from fine-tuning both backbones jointly rather than keeping them frozen.

V. CONCLUSION

We successfully implemented and evaluated convolutional neural networks for image classification on CIFAR-10 and

STL-10 datasets. Our custom CNN achieved 85.47% test accuracy on CIFAR-10, exceeding the assignment requirement of 82%. On STL-10, we demonstrated that pretrained models drastically outperform training from scratch, with ResNet18 achieving 93.15% accuracy using ImageNet weights compared to 62.6% when trained from scratch. The middle-fusion strategy combining ResNet18 and MobileNetV3-Small achieved 93.33% accuracy, the best performance among all evaluated approaches.

Key takeaways include: (1) computing normalization statistics directly from data ensures accurate preprocessing, (2) proper regularization (dropout, weight decay, label smoothing) is essential for training custom architectures and identifying the right hyperparameters prevents underfitting, (3) transfer learning provides massive performance gains with limited training data—30+ percentage point improvements, (4) architecture choice significantly impacts performance, with ResNet18 substantially outperforming MobileNetV3-Small on STL-10, and (5) fusion strategies can combine multiple models to achieve marginal improvements over the strongest individual component while leveraging complementary feature representations.

This assignment provided hands-on experience with CNN design, training optimization, transfer learning, and model fusion, reinforcing fundamental concepts in deep learning for computer vision.

Code Availability

The complete implementation, including all training scripts, model definitions, and evaluation code, is publicly available in our Google Colab notebook:

<https://colab.research.google.com/drive/1Rrj1FgM0I2p9t6K0ECumSJsWvIf8sQYn?usp=sharing>

The notebook contains reproducible code for all experiments reported in this paper, including data preprocessing, model training, hyperparameter tuning, and result visualization.